KV-Cache Oriented Query-Aware Sparse Attention Accelerator With Cross-Stage Precision-Configurable Digital CIM

Yang Zhang, Xilong Kang, Weixuan Wang, Yizhi Ding, Lizheng Ren, Yiran Zhang, Ruiqi Tan, Zhen Wang, Hao Cai, *Senior Member, IEEE*, and Bo Liu, *Member, IEEE*

***Abstract*—This brief proposes KV-CIM, a KV-Cache oriented Digital Compute-In-Memory (DCIM) sparse attention accelerator, to address computational and memory bottlenecks in autoregressive inference for large language models. Key innovations include: a) A query-aware pre-compute architecture, which dynamically selects and accesses KV-Cache for critical tokens at the pre-compute stage (Stage1) and deploys KV-Cache segmentally on memory-constrained edge devices while maintaining computational accuracy at the formal computation stage (Stage2); b) A cross-stage DCIM macro featuring precision-configurable adder trees, which works in approximate mode at Stage1 and changes to full precision mode at Stage2; c) A query-stationary dataflow that retains the current query tensors in q-CIM across stages to eliminate data movement. Under 28-nm CMOS technology, the proposed KV-CIM achieves 35.16 TOPS/W and 82% reduction of external memory access with negligible degradation in LLaMA2 expressiveness.**

***Index Terms*—Compute-in-memory based accelerator, KV-cache, sparse attention, precision-configurable DCIM.**

I. Introduction

THE TRANSFORMER architecture [1] in modern Large Language Models (LLMs) [2], [3], leverages self-attention for global context modeling and parallel processing efficiency. However, autoregressive inference in LLMs incurs significant computational overhead due to repeated attention over growing token sequences. The Key-Value Cache (KV-Cache) [4] mitigates this challenge by reusing cached Key-Value pairs, yet its continuous growth (up to 30% of model parameters [6]) strains resource-constrained devices. For long sequences (e.g., 2048+ tokens) and large batch sizes, KV-Cache access dominates latency [5], increasing external memory bandwidth demands (Challenge 1 in Fig. 1).

The quadratic complexity of dense attention limits Transformer scalability to long sequences and large models [5]. Sparse attention mitigates this by selectively attending to critical tokens, balancing memory-computation trade-offs. Recent dynamic sparsity (DS) techniques [7], [8], [9] further reduce self-attention latency by prioritizing tokens at runtime, concentrating resources on the attention computation of critical tokens (Challenge 2 in Fig. 1).

Digital Compute-In-Memory (DCIM) accelerators [11], [12], [20], [21] address the demands of self-attention in LLMs decoding by integrating memory and computation to minimize data movement. However, pre-compute sparse attention prediction incurs extra computation and area overhead [10], undermining DCIM efficiency (Challenge 3 in Fig. 1).

To address these challenges, this brief proposes KV-CIM, a KV-Cache oriented query-aware sparse attention accelerator. A sparse attention prediction mechanism is proposed to classify tokens to bypass non-critical token processing, reducing External Memory Access (EMA). A segmented KV-Cache access scheme is employed to mitigate off-chip memory demands and minimize EMA. Hardware implementation features a cross-stage DCIM architecture with precision-configurable adder trees.

The contributions of this brief are summarized as follows: a) We propose a query-aware pre-compute architecture, which predicts attention scores by evaluating Query against grouped critical features, selectively loading KV-Cache for critical tokens at Stage1, and deploys a segmented KV-Cache scheme with iterative safe-softmax updates for resource-limited devices at Stage2, reducing EMA by 82% with a 16 KB KV buffer; b) We propose a cross-stage DCIM macro featuring precision-configurable adder trees, which works in approximate mode at the pre-compute stage (Stage1) and switches to full precision mode at the formal computation stage (Stage2), enhancing hardware utilization across stages; c) We propose a query-stationary dataflow that retains Query tensors in q-CIM across Stage1 and Stage2 to eliminate inter-stage data movement; d) Evaluated on an industry 28-nm process technology, the proposed KV-CIM achieves 2.85× speedup in attention workload, and up to 1.51× improvement in energy efficiency compared to existing works.

II. KV-CIM Architecture and Workflow

The self-attention mechanism with KV-Cache, as illustrated in Fig. 1, is commonly used in LLMs [2], [3]. This brief introduces the KV-CIM architecture (Fig. 2) to address these

Received 25 February 2025; revised 6 May 2025; accepted 11 June 2025. Date of publication 17 June 2025; date of current version 1 August 2025. This work was supported by the National Key Research and Development Program of China under Grant 2023YFB4403103. This brief was recommended by Associate Editor J. Ding. *(Corresponding author: Bo Liu.)*

Yang Zhang, Xilong Kang, Weixuan Wang, Yizhi Ding, Lizheng Ren, Yiran Zhang, Ruiqi Tan, Hao Cai, and Bo Liu are with the School of Integrated Circuits, Southeast University, Nanjing 210096, China (e-mail: liubo_cnasic@seu.edu.cn).

Zhen Wang is with Nanjing Prochip Electronic Technology Co. Ltd., Nanjing 210001, China.

Digital Object Identifier 10.1109/TCSII.2025.3580135

1549-7747 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 29,2026 at 07:37:55 UTC from IEEE Xplore. Restrictions apply.

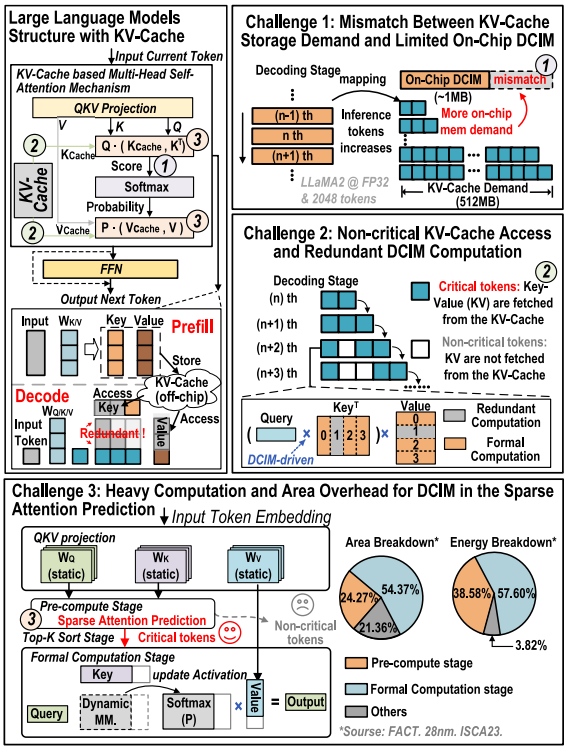


Fig. 1. A typical LLM accelerator architecture with KV-Cache and challenges.

emerging challenges posed by KV-Cache. KV-CIM introduces a query-aware pre-compute unit (detailed in Section II-A), which dynamically selects and loads critical tokens stored in off-chip DRAM based on the current Query tensor. Further, by employing a segmented KV-Cache access scheme (described in Section II-B) based on FlashAttention [5] algorithmic principles, KV-CIM enables a tiling scheme for KV-Cache deployment. In addition, we design a cross-stage precision-configurable DCIM (detailed in Section II-C) to leverage the computational characteristics of both Stage1 and Stage2, achieving energy-efficient attention computation with minimal loss of critical tokens. KV-CIM introduces a query-stationary dataflow (detailed in Section II-D). By retaining the current Query tensors in the q-CIMs across both Stage1 and Stage2, which eliminates repetitive Query reloading and dynamic updates, reducing cross-stage data movement.

Dynamic sparse attention [8] offers a solution by prioritizing critical tokens to reduce KV-Cache access induced by non-critical tokens. DS acceleration is comprised of three stages: (1) Pre-compute stage for token importance estimation by attention scoring, (2) Top-K selection to identify critical tokens, and (3) Formal computation stage executing attention operations exclusively on critical tokens.

As illustrated in Fig. 3, the KV-CIM workflow operates as follows: The Query (Q), Key (K), and Value (V) vectors, derived from W_Q, W_K, and W_V, are processed in parallel. The Query vectors are stored in the q-CIM macro, while Key and Value vectors remain in the on-chip buffer. These on-chip vectors are compared with the critical features of each Key-group loaded from the off-chip KV-Cache to update the critical feature dynamically. Under the control of the Dynamic Tokens

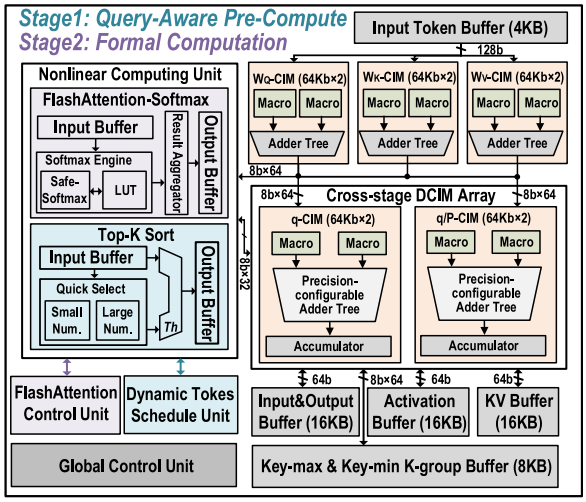


Fig. 2. Proposed KV-CIM architecture with cross-stage DCIM array.

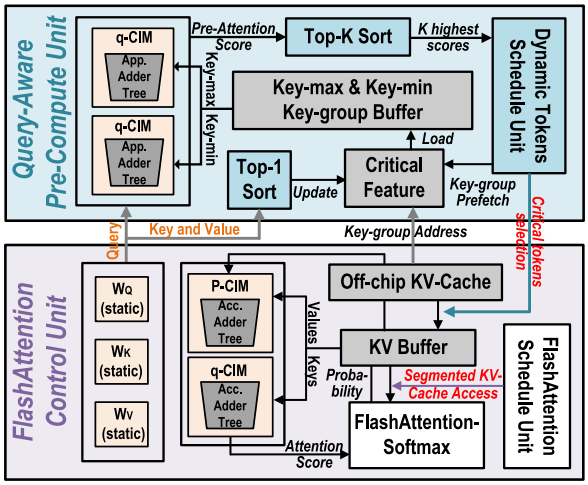


Fig. 3. The workflow of KV-CIM with query-aware pre-compute unit and FlashAttention control unit.

Schedule Unit, the updated feature values are transferred into the q-CIM to compute pre-attention scores, followed by Top-K sorting to select the K highest-scoring tokens (detailed in Section II-A). Corresponding critical token groups are then fetched from the off-chip KV-Cache into the on-chip KV Buffer. Finally, the FlashAttention Schedule Unit iteratively loads segmented Keys and Values from the KV Buffer to complete the attention through tile-wise processing, ensuring efficient resource utilization and minimal EMA.

III. KV-Cache Oriented Query-Aware Sparse Attention Accelerator

A. Query-Aware Pre-Compute Unit

Building on the PagedAttention algorithm [6] [13] for LLMs, we propose a Query-aware Pre-compute Unit (QPU), as shown in Fig. 4, which predicts attention scores by evaluating the current Query against grouped Key critical features, enabling the selection and loading of KV-Cache for critical tokens. The reconfigurable Top-K module identifies the top K largest elements with quick selection mechanism.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 29,2026 at 07:37:55 UTC from IEEE Xplore. Restrictions apply.

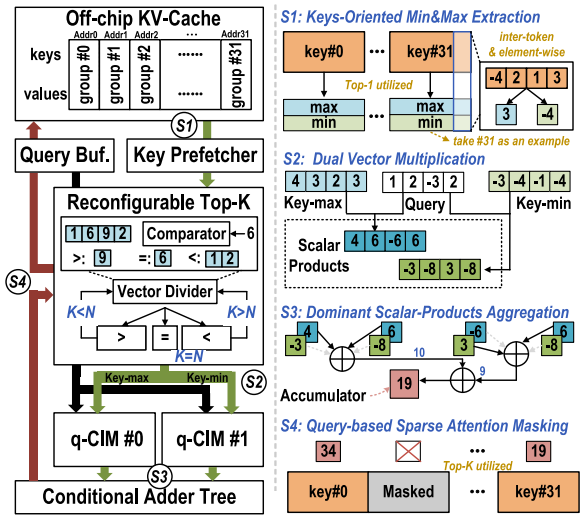


Fig. 4. The process of QPU and examples for selecting critical token groups.

Without sparse attention, the total memory access (TMA) for KV-Cache is:

$$TMA_{Cache} = N_{group} \times S_{group} \times \sum_{i=1}^{N} P_i \tag{1}$$

With QPU, the TMA is reduced to:

$$TMA_{QPU} = K \times S_{group} \times \sum_{i=1}^{K} P_i \tag{2}$$

N_{group} denotes the number of groups (N is set to 32 in this brief), S_{group} denotes the group size (S is set to 64), P_i indicates the effective KV-Cache utilization for group i.

In the pre-compute stage, KV-CIM utilizes on-chip memory with a Top-1 module to efficiently track the maximum and minimum values for each KV group, which employs fixed-size memory segments for KV groups to mitigate external memory overhead during Stage 1 (S1); In S2, the maximum and minimum values are multiplied by the current Query tensors in q-CIM; In S3, a conditional adder tree is employed to aggregate significant scalar products, specifically accumulating the maximum values obtained from the computations in S2. These accumulated values serve as the critical feature for each KV group, effectively summarizing the relevant information and improving the efficiency of subsequent selection processes; S4 then leverages a Top-K module to select and load the KV groups with the highest K scores from off-chip DRAM, optimizing memory access and focusing computational resources on KV-Cache for the most critical tokens.

Our analysis utilizing the MICSim [14] demonstrates improvements through the evaluation of prompt workloads across varying token lengths (64, 128, 256, 512, 1024). The QPU reduces KV-Cache access operations by 89% via dynamic computation for 2048-token sequences. Furthermore, QPU reduces energy consumption by 82% through optimized sparse attention mechanisms. Additionally, compared to SpARC [8], our work reduces off-chip memory access energy by 57%, computation cost by 56%, and inference latency by 19%. These results underscore the effectiveness of dynamic sparse attention in optimizing KV-Cache access.

B. FlashAttention Control Unit Based on Segmented KV-Cache Access Scheme

The softmax operation in attention mechanism necessitates a complete traversal of the entire KV-Cache, as mathematically formulated in equations, as is shown (3), (4), (5), (6). $m(x)$ is a scalar retained in on-chip memory through Top-1 selection (details in Section III-A, K=1), $f(x)$ represents the vectorized safe-softmax operation, while $l(x)$ denotes the summation over the resultant vectors, and $\text{softmax}(x)$ is a vector as the output of safe-softmax.

$$m(x) = \max_i(x_i) \tag{3}$$

$$f(x) = \left[e^{x_1 - m(x)}, \ldots, e^{x_i - m(x)}\right] \tag{4}$$

$$l(x) = \sum_i f(x)_i \tag{5}$$

$$\text{softmax}(x) = \frac{f(x)}{l(x)} \tag{6}$$

Due to the inherent instability of exponential computations, such as numerical overflow and decreased precision beyond a certain range, safe-softmax is often preferred in practical scenarios. However, as shown in (3), determining the maximum requires traversing all inputs, which involves extracting all Keys from the KV-Cache, computing the product with the current Query, and identifying the maximum. The large-batch max operation, combined with the summation in (5), imposes significant computational overhead and poses challenges for efficient hardware implementation.

To address the above, we implement the segmented KV-Cache access scheme by applying a tiling method that divides the KV-Cache into batch-wise segment, reducing on-chip SRAM demand without compromising computational precision. As shown in Fig. 5, this method partitions Key-Value pairs into four segments, allowing iterative updates of attention scores and output. The scheme restricts EMA to single-access attention computation during the inference stage, significantly alleviating pressure on limited on-chip resources. The KV-Cache segmentation matches the q-CIM macro capacity. During the pre-compute stage, q-CIMs process Key-max/min tensors from 32-group KV-Cache groups (64 elements/group). For FlashAttention, 32 tiling blocks (64 elements/tile) are employed at the formal computation stage. This brief introduced the FlashAttention Control Unit (FCU), which uses predefined DRAM addresses (aligned with Section III-A) to manage KV-Cache segmentation. By processing the current token's Query and the segmented KV-Cache, the FCU enables continuous updates of on-chip SRAM parameters, including Key, Value, max, sum, and output, to generate the final output. The segmented KV-Cache access scheme coupled with the FCU achieves a 65% reduction in latency while requiring only about 16 KB KV buffer.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 29,2026 at 07:37:55 UTC from IEEE Xplore. Restrictions apply.

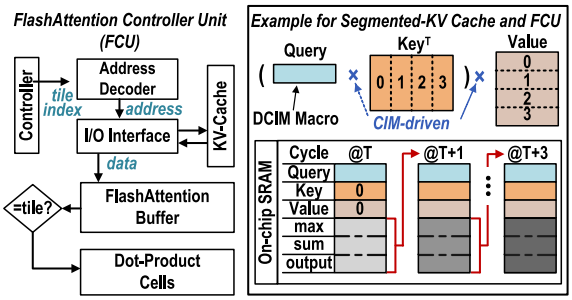


Fig. 5. An example of the segmented KV-Cache access scheme.

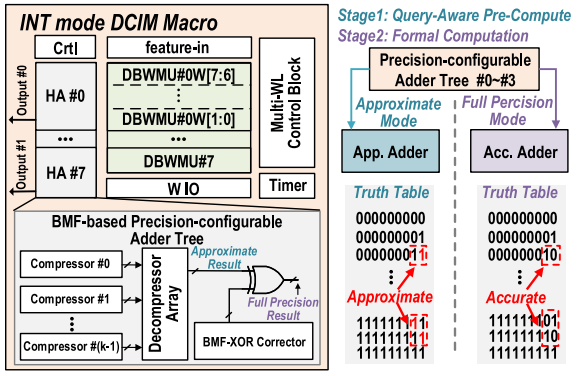


Fig. 6. Cross-stage precision-configurable digital CIM.

C. Cross-Stage Precision-Configurable Digital CIM

The pre-compute stage leverages relaxed precision constraints to prioritize computational efficiency through approximate processing techniques. In contrast, the formal computation stage requires high precision to maintain result accuracy. To meet these dual requirements, a Boolean Matrix Factorization (BMF) approximate logic synthesis approach [15], [16], [18] is employed to decompose truth tables, enabling efficient and flexible configurable circuit design, as illustrated in Fig. 6. Given the exponential truth table growth with input size, we partition an adder tree into subcircuits, each is approximated using BMF to create compressor and decompressor circuits. Compressors integrate logic gates and multiplexers, while decompressors use OR gate arrays. Multiplexers isolate compressor outputs, enabling full precision mode and approximate mode. For full precision mode, a sparse corrector circuit [15] is employed to compensate for approximation errors, ensuring accuracy with minimal computational overhead.

The cross-stage DCIM macro adapts dynamically, operating in approximate mode during the pre-compute stage and transitioning to full precision mode for the formal computation stage. The cross-stage precision-configurable DCIM, as is shown in Fig. 6, with its dual-mode capability, achieves Power-Delay-Product (PDP) reductions by 58.78% in approximate (app.) mode and 28.34% in full precision mode, compared to a conventional accurate (acc.) adder tree, as shown in Table I. This adaptive method delivers a DCIM macro that is specifically tailored for optimizing power efficiency and enhancing computational accuracy, thereby effectively addressing the demands of high-performance inference tasks at edge devices, while ensuring robustness and scalability.

D. Query-Stationary Dataflow

To address the limitations of DCIM for dynamic matrix multiplication, as highlighted in [20], we implement a query-stationary dataflow for q-CIMs. The Query tensors are replicated across multiple instances within the DCIM to enhance data reuse, thereby minimizing redundant data updates.

At the pre-compute stage, two q-CIMs compute Query $\times$ Key products. The intermediate softmax-generated P matrix is stored in P-CIM for subsequent P $\times$ Value computation at the formal computation stage. By retaining the current Query tensors in the q-CIM macro across both the pre-compute and formal computation stages, this design eliminates the demand for repetitive Query reloading and dynamic updates. Consequently, this approach reduces on-chip data movement by 28.20% compared to conventional dynamic dataflow methods. The dataflow enables query-stationary computation for grouped Key-Value pairs, seamlessly synergizing with the segmented KV-Cache access scheme. This allows for the iterative processing of tiles with a 16 KB on-chip buffer, further optimizing memory utilization and reducing latency.

TABLE I
COMPARISON OF ACCURATE ADDER TREE AND PRECISION-CONFIGURABLE ADDER TREE

	Acc. Adder Tree	Precision-configurable Adder Tree	
		App. Mode[*2]	Full Precision Mode[*3]
Delay[*1]	0.84 ns	0.59 ns	0.72 ns
Area[*1]	105.94 μm^2	80.56 μm^2	115.67 μm^2
Power[*1]	49.36 μW	28.96 μW	41.27 μW
PDP[*1]	41.46 ns-μW	17.09 ns-μW	29.71 ns-μW
(Reduction)	-	(58.78% $\downarrow$)	(28.34% $\downarrow$)

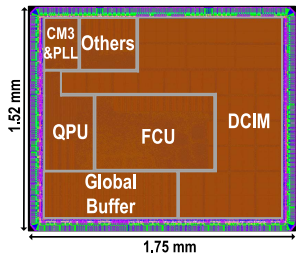
[*1] Synthesized using TSMC 28-nm under 0.8V, 200MHz.
[*2] Excluding compensation circuit.
[*3] Including compensation circuit.

IV. IMPLEMENTATION RESULTS

A dedicated reference model for system-level validation was first developed in MATLAB and underwent rigorous verification through numerical simulations. Subsequently, we leverage PyTorch to train the LLaMA2-7B model [3] using the AdamW optimizer for 50 epochs on the WikiText-2 dataset [17]. The digital portion of the KV-CIM is synthesized with the Design Compiler to estimate area and frequency, while layout-derived information regarding the CIM macro is concurrently extracted. Finally, we employ PrimeTime PX for power consumption analysis of the digital component at the model level, with corresponding metrics for the CIM macro obtained through post-layout simulation in Virtuoso.

Fig. 7 provides an analysis of the KV-CIM accelerator's hardware characteristics, highlighting the contributions of each module to the overall design metrics. Implemented in a 28-nm CMOS technology process, the KV-CIM achieves a

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 29,2026 at 07:37:55 UTC from IEEE Xplore. Restrictions apply.



Metric	This work
Technology	28nm CMOS
Layout Area	1.52*1.75 (mm^2)
Supply Voltage	0.6V-1.0V
Frequency	50MHz-200MHz
SRAM Size	60KB
Data Precision	INT8
Energy Efficiency	35.16 TOPS/W

Fig. 7. Layout of the prototype system and summary.

TABLE II
COMPARISON OF PROPOSED KV-CIM WITH STATE-OF-THE-ART WORKS

	TCAS-I'23 [19]	ISSCC'23 [20]	DAC'24 [21]	*This work*
Technique (nm)	28	28	28	28
Implementation	Digital CIM	Digital CIM	Digital CIM	Digital CIM
Die Area (mm^2)	2.00	3.93	2.92	2.66
Sparsity Level	Attention/FFN	Attention/Weight	Weight	Attention
SRAM Size (KB)	88	96	224	60
Supply Voltage (V)	0.9	0.64-1.03	-	0.6-1.0
Frequency (MHz)	50-200	20-320	500	50-200
Precision	INT8	INT8	INT8	INT8
Energy Efficiency (TOPS/W)	23.24 *1	25.22 *2, *3	28.20 *4	35.16 *5
Loss	< 1% (Accuracy) @ViT-Small	0.026 (BPC) @Daptive-Span	0.14% (Accuracy) @Longformer	-0.20 (Perplexity) @LLaMA2

*1 At 0.9V, 200MHz, under 0.65 pruning rate and ping-pong scheduling.
*2 Attention sparsity without KV-Cache = 92% for Enwik-8 task with 1024 tokens.
*3 Q/K/V Gen: 11.83 TOPS/W; $Q{\cdot}K^T$ & P·V: 53.83 TOPS/W.
*4 Bit-level sparsity@activation over 70%, modeling results: N = 8 and K = 4.
*5 Energy efficiency@attention sparsity with KV-Cache = 89% for WikiText-2 task with 2048 tokens.

total area of 2.66 mm^2 and an energy efficiency of 35.16 TOPS/W@INT8. The results indicate that query-aware pre-compute architecture and DCIM with precision-configurable adder trees yield reductions by 55.88% in area and 32.32% in energy consumption compared to FACT [10]. When benchmarked against state-of-the-art works under equivalent INT8 condition, the KV-CIM achieves 2.85× attention speedup and enhancement of 1.51×, 1.39×, and 1.25× in energy efficiency compared to P^3ViT [19], butterfly-network work [20], and Dyn-Bitpool [21], respectively.

V. CONCLUSION

This brief proposes KV-CIM, a pre-compute architecture integrating query-aware sparse attention acceleration which includes a Query-aware Pre-compute Unit (QPU) and a FlashAttention Control Unit (FCU). The QPU with evaluation for critical tokens identification reduces 89% non-critical KV-Cache load and off-chip memory access. The FCU enables a segmented KV-Cache access scheme through FlashAttention for small-batch processing and iterative on-chip SRAM updates, reducing the on-chip KV buffer to 16 KB without accuracy compression. To further optimize the architecture, we design the cross-stage DCIM array employing precision-configurable adder trees and query-stationary dataflow that approximately predict token importance at the pre-compute stage and smoothly transition to full-precision attention calculation at the formal computation stage. Compared to SpARC, our design achieves 57% lower off-chip memory energy, 56% reduced computation cost, and 19% latency improvement. Under INT8 quantization, KV-CIM achieves an energy efficiency improvement of up to 1.51× compared to previous works.

REFERENCES

[1] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2017, pp. 5998–6008.
[2] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
[3] H. Touvron et al., "LLAMA 2: Open foundation and fine-tuned chat models," 2023, *arXiv:2307.09288*.
[4] R. Pope et al., "Efficiently scaling transformer inference," in *Proc. Mach. Learn. Syst.*, vol. 5, 2023, pp. 606–624.
[5] T. Dao et al., "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 16344–16359.
[6] W. Kwon et al., "Efficient memory management for large language model serving with pagedattention," in *Proc. 29th Symp. Oper. Syst. Principles (SOSP)*, 2023, pp. 611–626.
[7] S. Dai, H. Genc, R. Venkatesan, and B. Khailany, "Efficient transformer inference with statically structured sparse attention," in *Proc. Design Autom. Conf. (DAC)*, 2023, pp. 1–6.
[8] H. Cho, D. Kim, S. Hwang, and J. Park, "SpARC: Token similarity-aware sparse attention transformer accelerator via row-wise clustering," in *Proc. Design Autom. Conf. (DAC)*, 2024, pp. 1–6.
[9] Z. Wang, G. Wang, H. Jiang, N. Xu, and G. He, "COSA: Co-operative systolic arrays for multi-head attention mechanism in neural network using hybrid data reuse and fusion methodologies," in *Proc. Design Autom. Conf. (DAC)*, 2023, pp. 1–6.
[10] Y. Qin et al., "FACT: FFN-attention co-optimized transformer architecture with eager correlation prediction," in *Proc. Int. Symp. Comput. Architect. (ISCA)*, 2023, pp. 1–14.
[11] R. Guo et al., "20.2 A 28nm 74.34TFLOPS/W BF16 heterogenous CIM-based accelerator exploiting denoising-similarity for diffusion models," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2024, pp. 362–364.
[12] F. Tu et al., "16.1 MuITCIM: A 28nm 2.24μJ/token attention-token-bit hybrid sparse digital CIM-based accelerator for multimodal transformers," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023, pp. 248–250.
[13] J. Tang, Y. Zhao, K. Zhu, G. Xiao, B. Kasikci, and S. Han, "Quest: Query-aware sparsity for efficient long-context LLM inference," 2024, *arXiv:2406.10774*.
[14] C. Wang, Z. Chen, and S. Huang, "MICSim: A modular simulator for mixed-signal compute-in-memory based AI accelerator," 2024, *arXiv:2409.14838*.
[15] B. Liu et al., "Layer-wise mixed-modes CNN processing architecture with double-stationary dataflow and dimension-reshape strategy," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 71, no. 10, pp. 4652–4664, Jun. 2024.
[16] S. Hashemi, H. Tann, and S. Reda, "BLASYS: Approximate logic synthesis using Boolean matrix factorization," in *Proc. Design Autom. Conf. (DAC)*, 2018, pp. 1–6.
[17] E. Frantar and D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot" in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 10323–10337.
[18] F. Avellaneda and R. Villemaire, "Delegation-relegation for Boolean matrix factorization," in *Proc. AAAI Conf. Artif. Intell. (AAAI)*, vol. 38, 2024, pp. 20632–20639.
[19] X. Fu et al., "P^3 ViT: A CIM-based high-utilization architecture with dynamic pruning and two-way ping-pong macro for vision transformer," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 4938–4948, Jun. 2023.
[20] S. Liu et al., "A 28nm 53.8TOPS/W 8b sparse transformer accelerator with in-memory butterfly zero skipper for unstructured-pruned NN and CIM-based local-attention-reusable engine," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2023, pp. 250–252.
[21] X. Xiang et al., "DYN-Bitpool: A two-sided sparse CIM accelerator featuring a balanced workload scheme and high CIM macro utilization," in *Proc. Design Autom. Conf. (DAC)*, 2024, pp. 1–6.

Authorized licensed use limited to: University of Electronic Science and Tech of China. Downloaded on October 29,2026 at 07:37:55 UTC from IEEE Xplore. Restrictions apply.